

ES -116

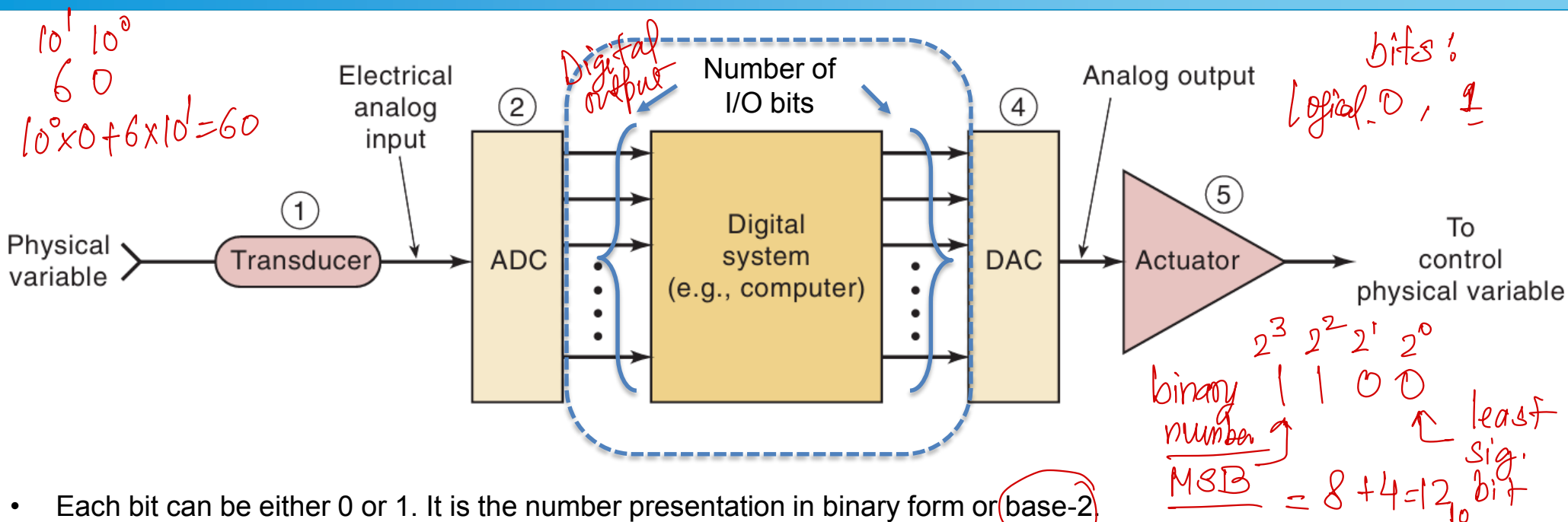
Principles and Applications of Electrical Engineering

Semester II, 2025-26

Welcome to the Digital World!

Reference book: Digital Systems Book by Tocci

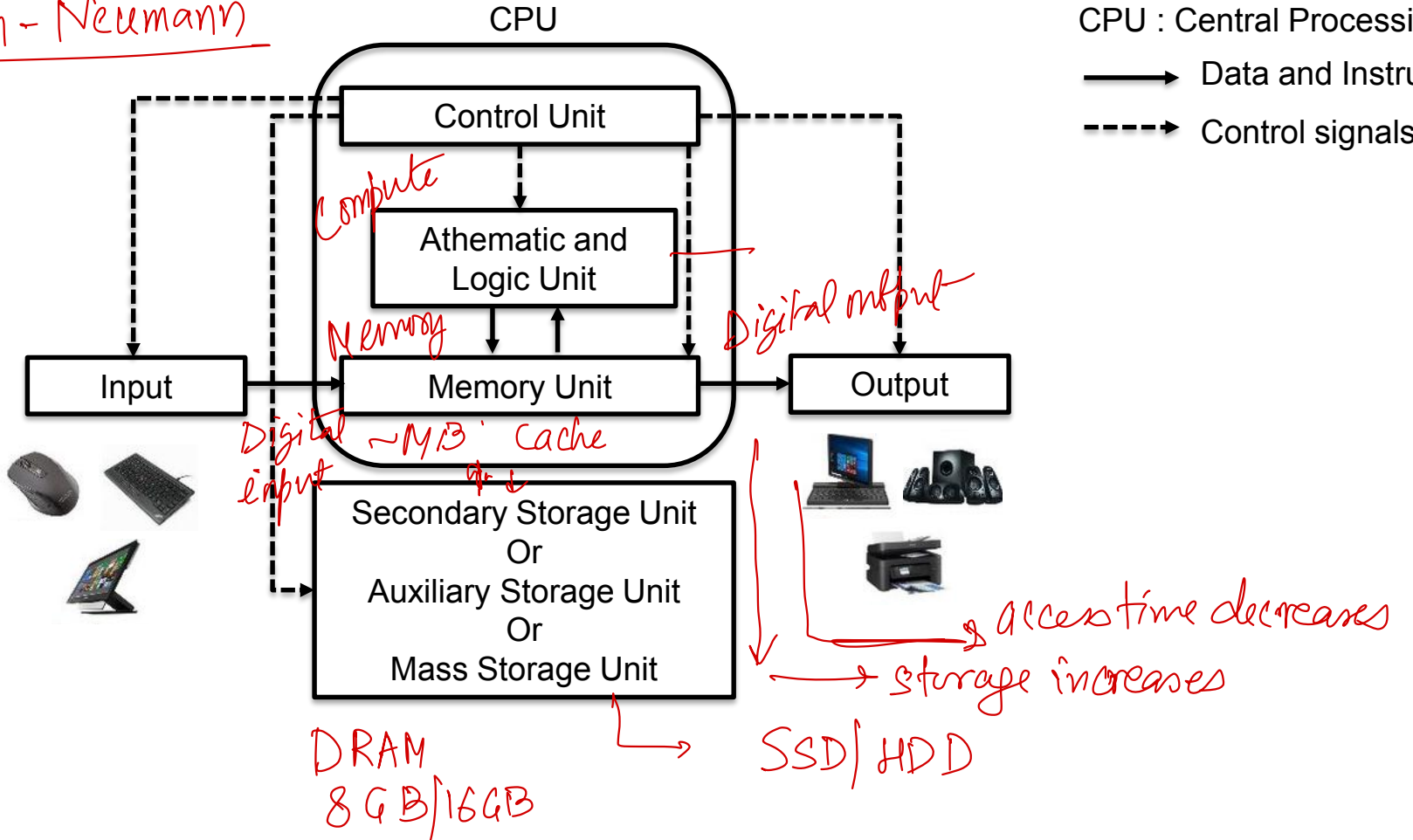
Recap



- Each bit can be either 0 or 1. It is the number presentation in binary form or base-2
- It is natural for hardware as the building blocks of digital system (switches) operates naturally between these states.
- Base-2 meaning is that bits from LSB (Least Significant Bit) to MSB (Most Significant Bit) carries a weight increasing from 2^0 to 2^{N-1} . Here, N is the total number of bits.
- Therefore, 11001 in base-2 will mean 25 in base-10 (i.e. decimal which natural for humans).

What is inside a Digital System (e.g. Computer)?

Von-Neumann



Binary Representation

- Data flow inside a computer is in binary numbers.
- Bit: 1/0
- Byte: 8 bits

Examples: Data storage capacity is expressed in KB, MB, GB, TB, PB, EB, ZB, YB.....

If it requires 1 transistor and 1 capacitor to store 1-bit, how many transistors are needed in a 8 GB RAM?

$$= 8 \times 1024^3$$

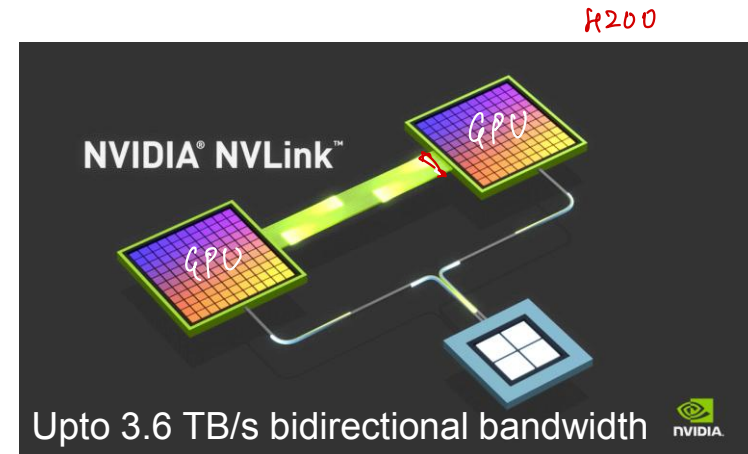
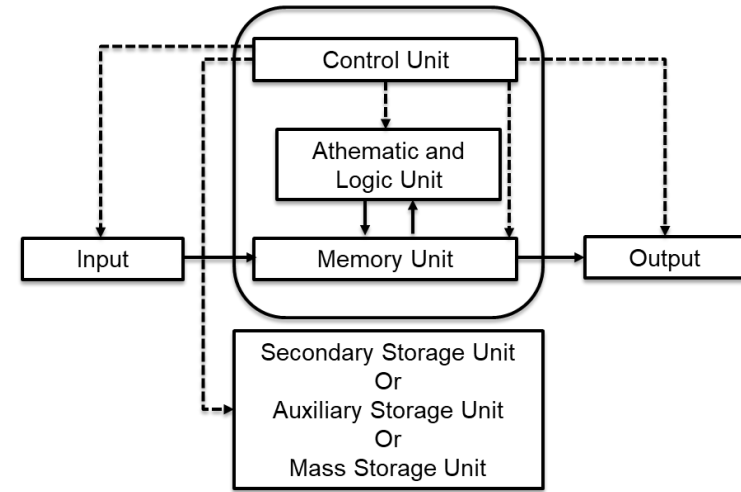
1024 Bytes 1024 MB
↑ ↑
1024 KB
[X] | DRAM (DDR)
[I] | 8GB/16GB - -

Examples: Data flow between Units is expressed in Bandwidth or throughput (KBps, MBps, GBps, TBps)

photon
→
Speed of light

~ 900 MBps
↳ 900 GBps

When you upgrade from a 10 Megabit fiber connection to a blazing fast 1 Gigabit fiber connection, does the data travel to your house faster?



How do Humans understand Computer Language?

Binary (base-2)

- Natural for hardware
- But long and hard for humans

Example: $25_{10} = 11001_2$

Hexadecimal (base-16)

- Compact way to represent binary
 - Easy for engineers

Example: $11001_2 = 19_{16}$

Hex is how engineers read binary without going crazy. Applications: Memory Addresses (e.g. $0x1A3F$), Color codes ($\#FF0000$), etc.

Number systems = language of representation

100110000001_2
 $(9\ 8\ 1)_{16}$
 2^4

Decimal (base-10)

- Natural for humans
- Not for hardware

Application: Displaying sensor values (temperature, voltage, speed), User inputs (keypads, GUIs)

What's this?

$A4:C3:F0:88:B1:22$
 $\xrightarrow{8\ bit}$ MAC ID (IP Address)

Possibly, you have seen in Arduino codes, as a shorthand for binary

Decimal	Hexadecimal	Binary
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Examples of converting number from one base to another

1000 (base 10) to _____ (binary or base 2)

1000 (base 10) to _____ (hex or base 16)

$16^3 16^2 16^1 16^0$
FFFF (hex) to $16^3 \times 15 + 16^2 \times 15 + 16^1 \times 15 + 16^0 \times 15$ (base 10)

E + F
10 - - - 15

FFFF (hex) to 1111 1111 1111 1111 (base 2)

From Decimal to any base: Successive division by the base

2) 1000 (← LSB
 500 0
 250 0
 125 0
 62 1
 31 0
 15 1
 7 1
 3 1
 1 1 ← MSB

1111101000
 512

16) 1000 (8 → 8
 62 14 → E
 3 → 3 → 3

0h x 3E8 ←
 0b x

Examples of converting number from one base to another

1000 (base 10) to _____ (binary or base 2)

1000 (base 10) to _____ (hex or base 16)

FFFF (hex) to _____ (base 10)

FFFF (hex) to _____ (base 2)

One Application: Interacting with Arduino

Q. Would you prefer another data representation?

Virtual Terminal

```
Parallel Data: 00000000000000000000000000000000
Parallel Data: 01010100000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
Parallel Data: 01010101000000000000000000000000
```

Q. What is the value addition from using Hex here?

Output Serial Monitor x

Message (Enter to send message to 'Arduino Uno' on '/dev/cu.usbmodem1101') New Line 115200 baud

```
=====+ Memory Size +=====
Maximum Static RAM Address: 0x8FF (2KB)
Maximum EEPROM Address:    0x3FF (1KB)
Maximum Flash Address:     0x7FFF (32KB)
===== Arduino EEPROM HEX Dump =====
```

Addr	0	1	2	3	4	5	6	7	
0x00	72	65	65	66	77	69	6E	67	reefwing
0x08	DC	05	DC	05	DC	05	DC	05	

Q. Would you use “delay(1000)” in your Arduino sketch or “delay(0x3E8)” or “delay(0b1111101000)”?

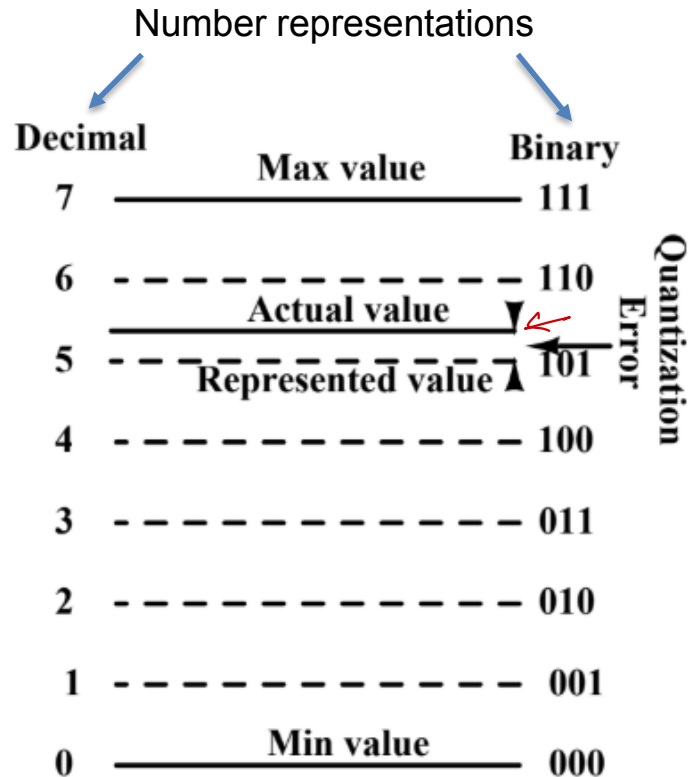
Question

Q. A small process-control computer uses hexadecimal codes to represent its 16-bit memory addresses.

- (a) How many hex digits are required? $\rightarrow 4 \rightarrow 16^4$
- (b) What is the range of addresses in hex? $\rightarrow 0000 - FFFF$
- (c) How many memory locations are there?

$$\downarrow \\ 2^{16}$$

Using Bits to represent a continuous value



- The difference between the actual value and the represented value is the quantization error.
- In the worst case, this error can be equal to half the interval width.
- If we divide the range of values into a larger number of intervals, each interval will be narrower and the quantization error will be smaller.
- However, the number of bits required for labeling the intervals will be more if we divide the range into a larger number of intervals.
- Notice that close but unequal values may be represented by the same digital number.
- Analog Value is a physical, real-world quantity (e.g. V, Kelvin or psi etc.)

$$\text{Analog Voltage} = (\text{Decimal Equivalent}) \times (V_{\text{LSB}})$$

Logical Quantity

Physical Quantity

What about fractions?

What will be stored binary value for this in your Arduino sketch?

int x = 10.10; $\rightarrow$ 1010
 $\rightarrow$ integer, whole number

How do we convert 10.10 from decimal to binary?


 1010.000110 --
 $2^5 2^4 2^3 2^2 2^1 2^0$

losing info.
→ less precision

$$\frac{1}{16}$$

more bits for fraction
→ more hardware

• 10×2

$2 \times 0.2 \rightarrow 0$

$2 \times 0.4 \rightarrow 0$

$2 \times 0.8 \rightarrow 0$

$2 \times 0.6 \rightarrow 1$

$2 \times 0.2 \rightarrow 1$

$0.4 \rightarrow 0$



binary fraction

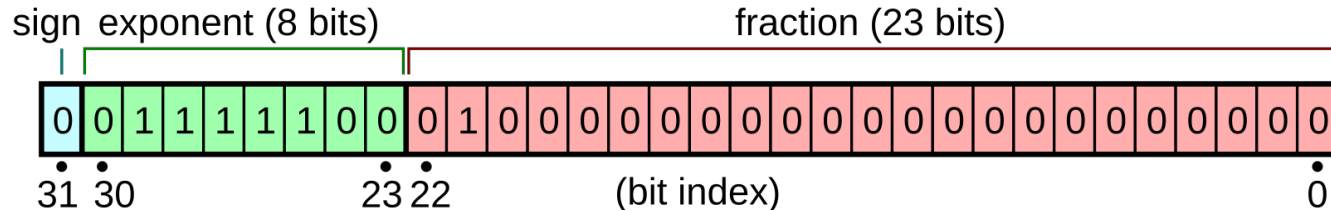
$\overbrace{1000}^{integer} . \underbrace{1000}_{fraction}$

$2^3 \quad 2^2 \quad 2^1 \quad 2^0 \quad 2^{-1} \quad 2^{-2} \quad 2^{-3} \quad 2^{-4}$

Decimal: 8.5

Application of Floating Point Numbers

Example of a FP32 format



value

$$= (-1)^{b_{31}} \times 2^{(b_{30}b_{29}\dots b_{23})_2 - 127} \times (1.b_{22}b_{21}\dots b_0)_2,$$

which yields

$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i} \right).$$

Why do we have this formats? → Precision

What is the downside of these formats? →

**FP4 vs. FP8 vs.
FP16 vs. FP32
for AI tasks**

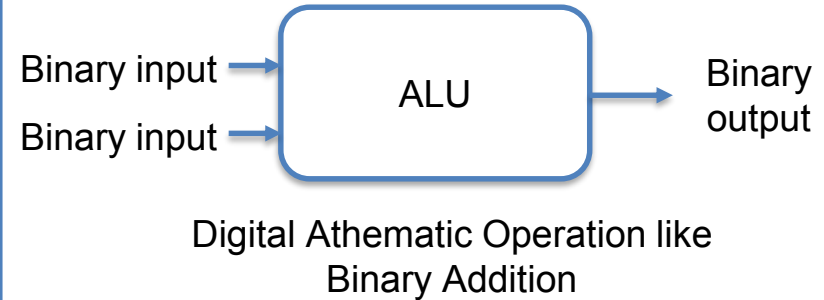
FP4	FP8	FP16	FP32
fastest/ most efficient	fast/ efficient	slower/ more demanding	slowest/ most demanding
fastest/ most efficient	fast efficient	more precise	most precise
fastest/ most efficient	less precise	more precise	slowest/ most demanding

In the Logical Realm...

In the Analog World



In the Digital World



Example:

Not a Analog quantity

$$\begin{array}{r} \begin{array}{cccccc} & 1 & 1 & 1 & 1 & & \\ & 0 & 1 & 1 & 0 & 1 & (13)_{10} \\ + & 1 & 1 & 0 & 1 & 1 & (27)_{10} \\ \hline & 1 & 0 & 1 & 0 & 0 & 0 & 40 \end{array} \\ \begin{array}{cccccc} \text{weight} & 32 & 16 & 8 & 4 & 2 & 1 \end{array} \end{array}$$

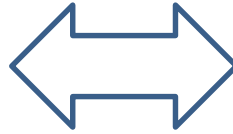
The diagram shows a binary addition example. The first number is 01101 (13 in decimal) and the second is 11011 (27 in decimal). The sum is 101000 (40 in decimal). The weights 32, 16, 8, 4, 2, 1 are written below the sum. A blue arrow points from the text "Not a Analog quantity" to the sum.

- Adding binary numbers is same as adding decimal number!
- i.e. adding the LSB (least significant bit) first
- Four different cases for binary addition:
 1. $0 + 0 = 0$
 2. $1 + 0 = 1$
 3. $1 + 1 = 0$, and carry '1' to the next position
 4. $1 + 1 + 1 = 1$, and carry '1' to the next position

How does computer do these arithmetic operations?

Decisions based on the logical inputs

1. $0 + 0 = 0$
2. $1 + 0 = 1$
3. $1 + 1 = 0$, and carry '1' to the next position
4. $1 + 1 + 1 = 1$, and carry '1' to the next position



$\&$, $\#1$, $\sim$
and , or , not

Logical operations

The binary number system uses only two digits, *1 and 0*, so it is perfect for representing logical relationships.

In logic, only two possible conditions exist for any input or output: **true and false**.

Practical examples

Logical operations (Examples)

A: You have/don't have a laddoo

B: Your friend has/doesn't have a laddoo

Y: I have/don't have a laddoo

Case I

I get a laddoo when either you have one laddoo *or* your friend has one.

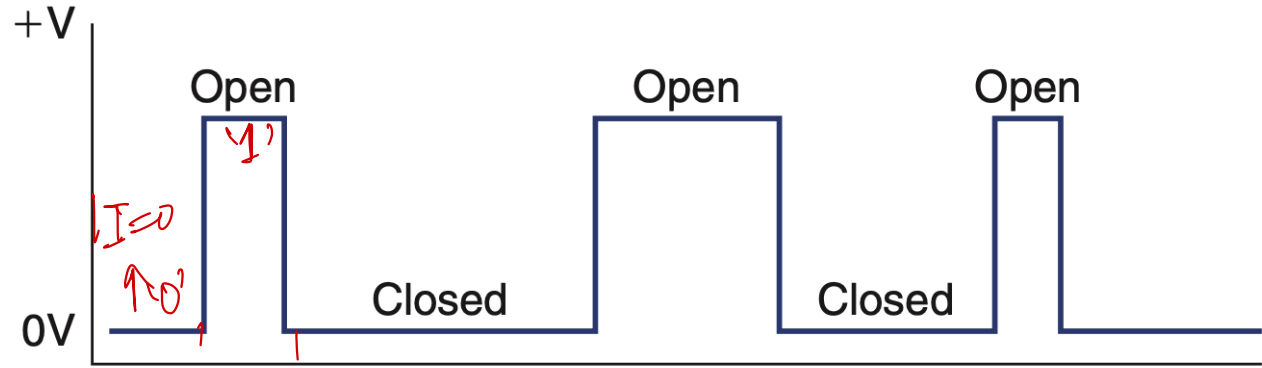
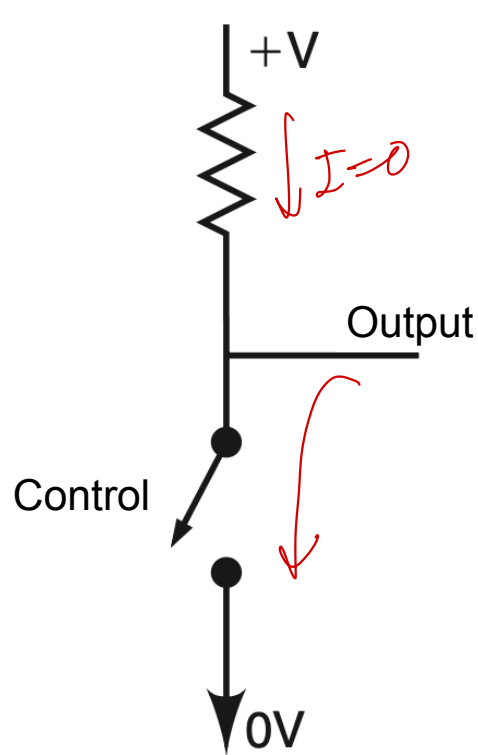
Case II

I get a laddoo when you have one laddoo *and* your friend has one.

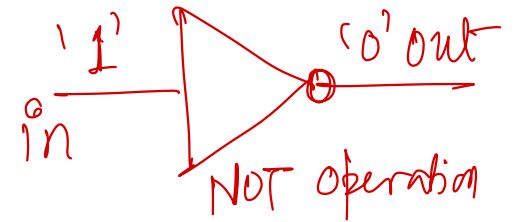
Case III

I get a laddoo only when you don't have one.

Simple implementation of a Logical Operation



For the given Output signal, draw the corresponding Control signal for the switch.



Boolean Algebra

Need

In logic, only two possible conditions exist for any input or output: **true and false**. The binary number system uses only two digits, *1 and 0*, so it is perfect for representing logical relationships

Origin

In 1854, a mathematician named George Boole wrote *An Investigation of the Laws of Thought*, in which he described the way we make logical decisions based on **true or false** circumstances. The methods he described are referred to today as **Boolean logic**, and the system of using symbols and operators to describe these decisions is called **Boolean algebra**.

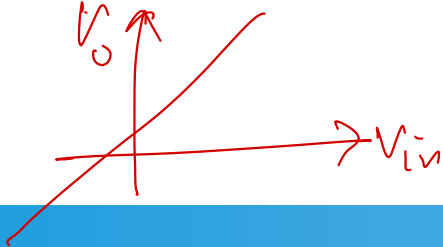
Formulation of a problem

Boolean algebra uses symbols to represent a logical expression that has one of two possible values: **true or false**. The main purpose of these logical expressions is to describe the relationship between a logic circuit's output (the decision) and its inputs (the circumstances).

Elementary algebra vs Boolean Algebra

Elementary Algebra

- An algebraic expression represents the relationship between input and output.
- Constants and variables can assume any numerical value.
- Include fractions, decimals, negative numbers, square roots, cube roots, logarithms, imaginary numbers, etc.
- The input-output relationship can be visualized by plotting the function (e.g. transfer characteristics).



Boolean Algebra

- Boolean algebra: a method to express the relationship between a logic circuit's inputs and outputs.
- Boolean constants and variables are allowed to have only two possible values, **0 or 1**.
- In Boolean algebra, there are only three basic operations: **OR, AND, and NOT**.
- The input-output relationship can be visualized using a truth-table.

2^n

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

OR

Truth tables

Truth table: A tool to visualize the input-output relationship

A	B	x
0	0	1
0	1	0
1	0	1
1	1	0

For one variable, how many distinct logic functions can we have?

A	Y
0	0
1	0

$Y=0$

A	Y
0	1
1	1

$Y=1$

A	Y
0	0
1	1

$Y=A$

A	Y
0	1
1	0

$Y=\bar{A}$ NOT

- How many distinct logic functions are possible for n input variables?
- The truth table must enumerate all possible combinations of input variables. So it will have 2^n rows.
- The function value can be either 0 or 1 in each row. Thus there are 2 independent choices per row.
- So the number of distinct truth tables is = $2^{\text{no. of rows}} = 2^{2^n}$.
- Each distinct truth table represents a different logic function.
(Though some of these functions could be trivial).

Truth tables for 2 input variables

- For 2 input variables, How many distinct logic functions can we have?

$$\rightarrow 2^4 = 16$$

- For 2 input variables, some logic functions are:

A	B	Y_1 OR	Y_2 NOR	Y_3 AND	Y_4 NAND	Y_5 XOR	Y_6 XNOR
0	0	0	1	0	1	0	1
0	1	1	0	0	1	1	0
1	0	1	0	0	1	1	0
1	1	1	0	1	0	0	1

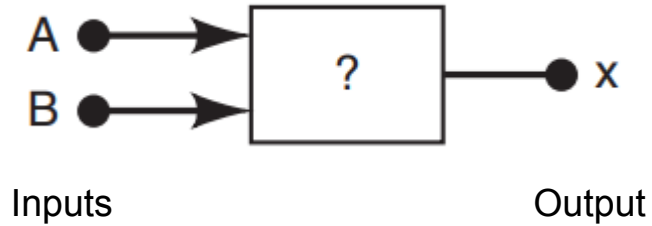
$$Y_2 = \overline{Y_1}$$
$$Y_4 = \overline{Y_3}$$

:

- The output of OR is '1' if either or both inputs are '1'.
- The output for the AND function is '1' only if both inputs are '1'.
- The output for the XOR (exclusive OR) function is '1' only if exactly one of the inputs is '1'.
- It is like the OR function but excludes the case when both inputs are '1' – hence the name.
- NOR, NAND and XNOR are inverted values of OR, AND and XOR respectively.

Truth tables

Block diagram



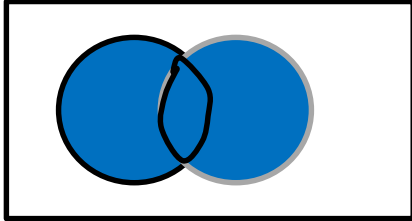
Truth table: A tool to visualize the input-output relationship

A	B	x
0	0	1
0	1	0
1	0	1
1	1	0

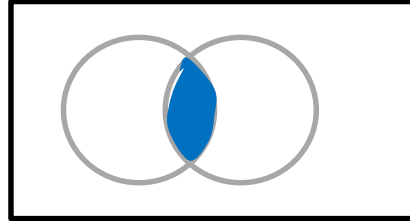
Draw a truth table for 3 input and 1 output block.

Boolean algebra basic operations

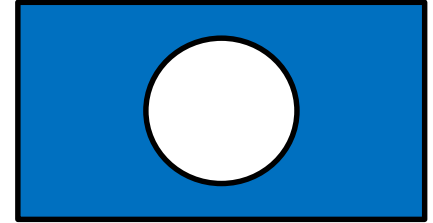
OR operation



AND operation



NOT operation



Truth tables

Mathematical
expression

Symbol

Boolean algebra laws

OR laws

- $A + 0 = A$
- $A + 1 = 1$
- $A + A = A$
- $A + \bar{A} = 1$



Commutative law

- $A + B = B + A$
- $A \cdot B = B \cdot A$

AND laws

- $A \cdot 0 = 0$
- $A \cdot 1 = A$
- $A \cdot A = A$
- $A \cdot \bar{A} = 0$



Associative law

- $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
- $(A + B) + C = A + (B + C)$

Inversion law

- $\bar{\bar{A}} = A$

Distributive law

- $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
- $A + (B \cdot C) = (A + B) \cdot (A + C)$

Boolean algebra theorems

De Morgan's rules or theorems

1) The first theorem states that the complement of the product of the variables is equal to the sum of their individual complements of a variable.

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

2) The second theorem states that the complement of the sum of variables is equal to the product of their individual complements of a variable.

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

Simple check: (Truth tables)

A complex logical decision

The airplane's computer is constantly monitoring three specific physical sensors. We will assign them our Boolean variables:

G (Gear Status): 1 = The landing gear is safely DOWN and locked.

0 = The gear is UP inside the plane.

A (Altitude): 1 = The plane is dangerously LOW to the ground.

0 = The plane is safely high in the sky.

T (Throttle): 1 = The pilot has pulled the engines back to IDLE (slowing down).

0 = The engines are pushing high power.

expression $A + \bar{A}B = A + B$?

law $A + B \cdot C = (A + B)(A + C)$

$\Rightarrow A + \bar{A}B = (A + \bar{A})(A + B)$
 $= \underline{1 \cdot (A + B)}$

To alert the pilot if they are about to crash: Sound the warning horn **IF** the landing gear is currently UP, **AND** the pilot is showing signs of trying to land (which means they are either flying very LOW, **OR** they have pulled the throttles to IDLE).

$$Y = \bar{G} \cdot (A + T)$$

$$\Rightarrow Y = \bar{G} \cdot (A + \bar{A}T)$$

$$\Rightarrow Y = \bar{G} \cdot (A + T)$$

$$Y = m_1 + m_2 + m_3$$

$$= \bar{G}\bar{A}T + \bar{G}A\bar{T} + \bar{G}AT$$

$$= \bar{G}(\bar{A}T + A\bar{T} + AT)$$

$$= \bar{G}(\bar{A}T + A(T + \bar{T})) = \bar{G}(A + \bar{A}T)$$

	(inputs)			(outputs)
	G	A	T	Y
m ₀	0	0	0	0
m ₁	0	0	1	1
m ₂	0	1	0	1
m ₃	0	1	1	1
m ₄	1	0	0	0
m ₅	1	0	1	0
m ₆	1	1	0	0
m ₇	1	1	1	0

From Truth Tables to Logical Expressions

- Every truth table is equivalent to a logic expression.
- How do we associate a given truth table with a logic expression?
- A truth table must enumerate all possible combinations of logic values for the input variables.
- Select all rows which have a '1' as the function value.
- If the input combination falls in any of these rows, the function evaluates to '1'.
- Therefore the output must be the OR of all these rows.
- But how do we associate a row of the truth table with a logic expression?

Examples

A	Y
0	1
1	0

$Y = \bar{A}$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

$Y = \bar{A}B + A\bar{B} + AB$
 $= \bar{A}B + A = A + B$

A	B	Y
0	0	0
0	1	0
1	1	1
1	0	0

$Y = AB$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$Y = \bar{A}B + A\bar{B}$

From Truth Tables to Logical Expressions

Each row of the truth table may be represented by an AND function of each of the input variables or their complements.

- Consider a truth table for 3 input variables A , B and C . A particular row may contain the values 1 0 1 for these three inputs.
These values uniquely identify this row of the truth table.
- We know that the AND of three Boolean variables can be '1' only if **all** of them are '1'.
(If any one is '0', the product would be '0').
- So the row is characterised by the AND of all variables whose value is '1' and the complement of all variables whose value is '0' in this row.
- Thus this row would be represented as $A \cdot \bar{B} \cdot C$.

Identification of rows by the product of all input variables or their complements (as used for uniquely identifying a row of the truth table) is called a **minterm**.

From Truth Tables to Logical Expressions

The logic function is an OR of all those minterms for which the function evaluates to '1'.

Let us illustrate it with an example with 3 input variables. We show two independent logic functions F1 and F2 of these 3 variables.

A	B	C	F1	F2
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The first function has the value '1' for rows with values of A, B, C as 001, 010, 011 and 111.

So the function is:

$$\overline{A} \cdot \overline{B} \cdot C + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot B \cdot C + A \cdot B \cdot C.$$

The second function is '1' for ABC values of 000, 010, 011, 101, 110 and 111.

Correspondingly it can be written as

$$\overline{A} \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot B \cdot C + A \cdot \overline{B} \cdot C + A \cdot B \cdot \overline{C} + A \cdot B \cdot C$$

From Truth Tables to Logical Expressions

This method of associating a truth table with a logical expression is general and can be used with any number of input variables.

- A logical expression which identifies a row of the truth table as a product is called a “**minterm**”. Each “minterm” is the product of *all* input variables – either in their original form or in the complement form.
- The final logic function representing the truth table is then the OR (or sum) of all “minterms” for which the function has the value ‘1’.
- The method of expressing the logic function using minterms is a standard (or **canonical**) form which is also known as a **sum of products** form.
- A minterm expression for the i ’th row in a truth table is represented as $\mathbf{m}_i$.

Canonical: which follows canons or well defined rules.

From Truth Tables to Logical Expressions

Recall that we had said that we can represent numbers using binary digits (bits) by allocating place values to these bits which are powers of 2, just like decimal numbers. Now consider the functions F1 and F2 of 3 variables which we had used as examples earlier.

A	B	C	F1	F2
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- The rows for which the output of the first function is '1' are: 001, 010, 011 and 111.
- If we interpret the row as a decimal number derived with place values which are powers of 2, we can identify these rows as no. 1, 2, 3 and 7.
- Using this convention, we can also write the function as $m_1 + m_2 + m_3 + m_7$, where m_i represents the i 'th minterm.

Question

Imagine three judges (A, B, and C) voting on a talent show. They push a button to vote "Yes" (1) or "No" (0). The stage light (L) only turns green (1) if a **majority** (two or more judges) vote Yes.

(inputs) (outputs)

A	B	C	
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Question

Imagine three judges (A, B, and C) voting on a talent show. They push a button to vote "Yes" (1) or "No" (0). The stage light (L) only turns green (1) if a majority (two or more judges) vote Yes.

$$Y = 1$$

	(inputs)			(outputs)
	A	B	C	Y
m_0	0	0	0	0
m_1	0	0	1	0
m_2	0	1	0	0
m_3	0	1	1	1
m_4	1	0	0	0
m_5	1	0	1	1
m_6	1	1	0	1
m_7	1	1	1	1

$$\begin{aligned} Y &= m_3 + m_5 + m_6 + m_7 \\ &= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC \\ &= \bar{A}BC + A\bar{B}C + AB(C + \bar{C}) \\ &= \bar{A}BC + A(B + \bar{B}C) \quad \leftarrow 1 \\ &= \bar{A}BC + AB + AC \quad \leftarrow B + C \\ &= B(A + \bar{A}C) + AC \\ &= B(A + C) + AC \end{aligned}$$

More than
two judges.

$$\boxed{Y = AB + BC + AC}$$

What if Output looks like this?

(inputs) (outputs)

	A	B	C	Y
m_0	0	0	0	0
m_1	0	0	1	1
m_2	0	1	0	1
m_3	0	1	1	1
m_4	1	0	0	1
m_5	1	0	1	1
m_6	1	1	0	1
m_7	1	1	1	1

alternative $\bar{Y} = \bar{A} \cdot \bar{B} \cdot \bar{C} \Rightarrow Y = \overline{\bar{A} \cdot \bar{B} \cdot \bar{C}} = A + B + C$

$Y = m_1 + \dots + m_7$
 $= \bar{A}\bar{B}C + \dots + ABC$

Is there an alternative way for logic expression?

(inputs)			(outputs)
A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

→ $\bar{Y} = \bar{A}\bar{B}\bar{C} \Rightarrow Y = A + B + C$
i.e. Product of sum.
 $Y = (\overset{\uparrow}{A} + \overset{\uparrow}{B} + \overset{\uparrow}{C})$
bar of inputs

From Truth Tables to Logical Expressions

We had remarked on the duality of theorems in Boolean logic. That provides us with another way of associating a logic expression with a truth table.

Select all rows which have a '0' as the result. If the input combination falls in *any* of these rows, the output is '0'. The output must be the AND of all these rows.

How do we uniquely represent a row for which the function evaluates to '0'?

- Consider again a truth table for 3 input variables A , B , and C , with a row which contains the values 1 0 1 for these three inputs.
- We know that the OR of three Boolean variables can be '0' only if **all** of them are '0'. (If any one is '1', the 'sum' would be '1').
- So the row is characterised by the OR of all input variables whose value is '0' and the complement of all input variables whose value is '1' in this row.
- So this row would be represented as $\overline{A} + B + \overline{C}$.

From Truth Tables to Logical Expressions

This alternative way of finding a logical expression to represent a row of the truth table for which the function evaluates to '0' is called a **Maxterm** and represented as **M_i** .

Let us use the same example as the one we had taken earlier (with a truth table for two functions F1 and F2 of three input variables).

A	B	C	F1	F2
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The rows for which the value of the first function (F1) is '0' are: 000, 100, 101 and 110.

These are represented by Maxterms:
 $A + B + C$, $\bar{A} + B + C$, $\bar{A} + B + \bar{C}$, and
 $\bar{A} + \bar{B} + C$, respectively.

So, F1 can be represented in the alternative form as:

$$(A + B + C) \cdot (\bar{A} + B + C)(\bar{A} + B + \bar{C}) \cdot (\bar{A} + \bar{B} + C)$$

From Truth Tables to Logical Expressions

For the function F2, we have:

A	B	C	F1	F2
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The rows for which the value of the second function (F2) is '0' are: 001 and 100.

These are represented by Maxterms:
 $A + B + \overline{C}$ and $\overline{A} + B + C$.

So, F2 can be represented in the alternative form as:

$$(A + B + \overline{C}) \cdot (\overline{A} + B + C)$$

This is also a **canonical** form and is known as the **product of sums** form.

Question

You are planning a weekend road trip. Your default attitude is optimistic: The trip is happening and it will be fun (Output = 1).

There are three friends who might come: Aditya (A), Bhumi (B), and Chirag (C). A '1' means they are in the car; a '0' means they stayed home.

The trip is going to be a success *unless* one of two specific, disastrous social dynamics happens. Let's look at the truth table:

(inputs) (outputs)

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$Y = (A + \bar{B} + \bar{C}) \cdot (\bar{A} + B + C)$$